Internationalization in Java

Senior Level: Architecture, Security and Delivery

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

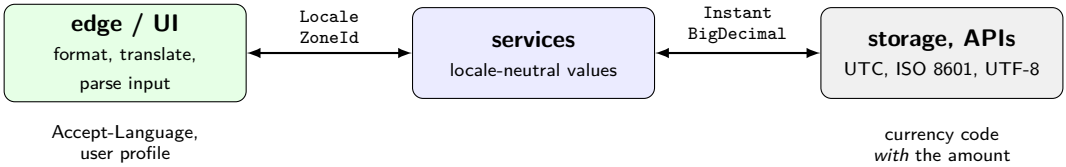
v2026-10-15

Outline

1. Architecture
2. Bundles at Scale
3. Security
4. Delivery and Operations
5. Summary

Architecture

Where internationalization lives in a system



- Inside the system everything is **locale-neutral**: `Instant`, `BigDecimal` + ISO currency code, keys instead of texts.
- Only the **edge** formats and parses for a person; the locale and zone travel with the request, never as a global default.
- Logs, files, JSON, CSV, database: `Locale.ROOT`, UTF-8, ISO 8601, UTC.

Language negotiation

Algorithm 1: LOOKUP(R, S) (RFC 4647, `Locale.lookup`)

Input: ranges $R = \langle r_1, \ldots, r_n \rangle$ sorted by weight q; supported locales S
Output: the best supported locale, or none

```
1 foreach range r in R with q > 0 do
2     t ← r
3     while t is not empty do
4         if some s ∈ S equals t then return s
5         remove the last subtag of t                    // de-CH → de
6 return none
```

Accept-Language in Java

codeoutput/Negotiate.java — Java

```
5  public class Negotiate {
6      public static void main(String[] args) {
7          List<Locale> supported = List.of(Locale.ENGLISH, Locale.of("tr"), Locale.GERMAN);
8          String[] headers = {                    // HTTP Accept-Language from browsers
9              "tr-TR,tr;q=0.9,en-US;q=0.8,en;q=0.7",
10             "de-CH,fr;q=0.8",
11             "fr-FR,fr;q=0.9,en;q=0.5",
12             "ja-JP",
13             "*;q=0.1,de;q=0.5"};
14         for (String h : headers) {
15             List<LanguageRange> wanted = LanguageRange.parse(h);
16             Locale best = Locale.lookup(wanted, supported);       // RFC 4647 lookup
17             System.out.printf("%-38s -> %s%n", h, best == null ? "(none: use the default)" : best);
18         }
19     }
20 }
```

`de-CH` falls back to `de`; French is not offered, so `en` wins; `ja-JP`: nothing, use the default.

Accept-Language in Java

codeoutput/Negotiate.java — Java

```java
public class Negotiate {
    public static void main(String[] args) {
        List<Locale> supported = List.of(Locale.ENGLISH, Locale.of("tr"), Locale.GERMAN);
        String[] headers = {                    // HTTP Accept-Language from browsers
            "tr-TR,tr;q=0.9,en-US;q=0.8,en;q=0.7",
            "de-CH,fr;q=0.8",
            "fr-FR,fr;q=0.9,en;q=0.5",
            "ja-JP",
            "*;q=0.1,de;q=0.5"};
        for (String h : headers) {
            List<LanguageRange> wanted = LanguageRange.parse(h);
            Locale best = Locale.lookup(wanted, supported);       // RFC 4647 lookup
            System.out.printf("%-38s -> %s%n", h, best == null ? "(none: use the default)" : best);
        }
    }
}
```

Terminal — codeoutput/Negotiate.txt

```
tr-TR,tr;q=0.9,en-US;q=0.8,en;q=0.7    -> tr
de-CH,fr;q=0.8                         -> de
fr-FR,fr;q=0.9,en;q=0.5                -> en
ja-JP                                  -> (none: use the default)
*;q=0.1,de;q=0.5                       -> de
```

`de-CH` falls back to `de`; French is not offered, so `en` wins; `ja-JP`: nothing, use the default.

Bundles at Scale

The application module

codeoutput/LanguagePack/src/app/module-info.java — Java

```
module app {
    exports com.app.i18n.spi;
    uses com.app.i18n.spi.MessagesProvider;      // language packs may provide bundles
}
```

codeoutput/LanguagePack/src/app/com/app/i18n/spi/MessagesProvider.java — Java

```
package com.app.i18n.spi;

import java.util.spi.ResourceBundleProvider;

// name required by getBundle: <package of base name>.spi.<Name>Provider
public interface MessagesProvider extends ResourceBundleProvider { }
```

codeoutput/LanguagePack/src/app/com/app/Main.java — Java

```
public class Main {
    public static void main(String[] args) {
        ResourceBundle rb = ResourceBundle.getBundle("com.app.i18n.Messages",
                                                     Locale.forLanguageTag(args[0]));
        System.out.println("[" + rb.getLocale() + "] " + rb.getString("hello"));
    }
}
```

The language pack module

codeoutput/LanguagePack/src/app.tr/module-info.java — Java

```
module app.tr {
    requires app;
    provides com.app.i18n.spi.MessagesProvider with com.app.tr.TurkishMessages;
}
```

codeoutput/LanguagePack/src/app.tr/com/app/tr/TurkishMessages.java — Java

```
package com.app.tr;

import com.app.i18n.spi.MessagesProvider;
import java.util.Locale;
import java.util.spi.AbstractResourceBundleProvider;

public class TurkishMessages extends AbstractResourceBundleProvider
                             implements MessagesProvider {
    @Override
    protected String toBundleName(String baseName, Locale locale) {
        return "com.app.tr.Messages_" + locale.getLanguage();  // own package: no split
    }
}
```

A pack is a module that **provides** the service the application **uses**.

Adding a language without changing the application

Terminal codeoutput/LanguagePack.txt

```
$ find src -type f | sort
src/app/com/app/i18n/Messages.properties
src/app/com/app/i18n/spi/MessagesProvider.java
src/app/com/app/Main.java
src/app/module-info.java
src/app.tr/com/app/tr/Messages_tr.properties
src/app.tr/com/app/tr/TurkishMessages.java
src/app.tr/module-info.java
$ javac -d out --module-source-path src -m app,app.tr
$ java -p out/app -m app/com.app.Main tr
[] Hello from the app module
$ java -p out/app:out/app.tr -m app/com.app.Main tr
[tr] Türkçe dil paketinden merhaba
```

In a named module `getBundle` also asks the `ResourceBundleProvider` services. The Turkish pack is a separate module on the module path: present, Turkish; absent, English. Two modules may not share a package, so the pack uses its own.

Bundles from a folder, reloaded

codeoutput/Reload/Reload.java — Java

```
12      static class FolderControl extends ResourceBundle.Control {
13          public List<String> getFormats(String base) { return FORMAT_PROPERTIES; }
14          public long getTimeToLive(String base, Locale l) { return 0; }      // check
            every time
15          public boolean needsReload(String b, Locale l, String f, ClassLoader c,
16                                     ResourceBundle old, long loaded) { return true; }
17          public ResourceBundle newBundle(String base, Locale l, String format,
18                                          ClassLoader c, boolean reload) throws
            IOException {
19              Path p = Path.of("texts", toBundleName(base, l) + ".properties");
20              if (!Files.exists(p)) return null;
21              try (Reader r = Files.newBufferedReader(p)) { return new
            PropertyResourceBundle(r); }
22          }
23      }
24      public static void main(String[] args) throws IOException {
25          FolderControl control = new FolderControl();
26          Locale tr = Locale.of("tr");
27          System.out.println(ResourceBundle.getBundle("Texts", tr, control).getString("
            hi"));
28          Files.writeString(Path.of("texts/Texts_tr.properties"), "hi=Selam\n");  // a
            translator edits
29          System.out.println(ResourceBundle.getBundle("Texts", tr, control).getString("
            hi"));
30      }
31  }
```

Bundles from a folder, reloaded

codeoutput/Reload/Reload.java — Java

```
12      static class FolderControl extends ResourceBundle.Control {
13          public List<String> getFormats(String base) { return FORMAT_PROPERTIES; }
14          public long getTimeToLive(String base, Locale l) { return 0; }      // check
            every time
15          public boolean needsReload(String b, Locale l, String f, ClassLoader c,
16                                     ResourceBundle old, long loaded) { return true; }
17          public ResourceBundle newBundle(String base, Locale l, String format,
18                                          ClassLoader c, boolean reload) throws
            IOException {
19              Path p = Path.of("texts", toBundleName(base, l) + ".properties");
20              if (!Files.exists(p)) return null;
21              try (Reader r = Files.newBufferedReader(p)) { return new
            PropertyResourceBundle(r); }
22          }
23      }
24      public static void main(String[] args) throws IOException {
25          FolderControl control = new FolderControl();
26          Locale tr = Locale.of("tr");
27          System.out.println(ResourceBundle.getBundle("Texts", tr, control).getString("
            hi"));
28          Files.writeString(Path.of("texts/Texts_tr.properties"), "hi=Selam\n");  // a
            translator edits
29          System.out.println(ResourceBundle.getBundle("Texts", tr, control).getString("
            hi"));
30      }
31  }
```

Terminal — codeoutput/Reload.txt

```
$ cat texts/Texts_tr.properties
hi=Merhaba
$ java Reload.java
Merhaba
Selam
```

A `Control` decides *where* bundles come from (files, database, a translation server) and *how long* they are cached. Translators can fix texts without a new release.

Three bundles, three mistakes

codeoutput/BundleCheck/Messages.properties **properties**

```
title=Order summary
items={0} items in {1}
bye=Goodbye
```

codeoutput/BundleCheck/Messages_tr.properties **properties**

```
title=Sipariş özeti
items={0} ürün
```

codeoutput/BundleCheck/Messages_de.properties **properties**

```
title=Bestellübersicht
items={0} Artikel in {1}
bye=Auf Wiedersehen
extra=nicht in der Basisdatei
```

Can you find them?

Three bundles, three mistakes

codeoutput/BundleCheck/Messages.properties **properties**

```
title=Order summary
items={0} items in {1}
bye=Goodbye
```

codeoutput/BundleCheck/Messages_tr.properties **properties**

```
title=Sipariş özeti
items={0} ürün
```

codeoutput/BundleCheck/Messages_de.properties **properties**

```
title=Bestellübersicht
items={0} Artikel in {1}
bye=Auf Wiedersehen
extra=nicht in der Basisdatei
```

Can you find them? German has an unknown key, Turkish misses `bye`, and Turkish `items` lost `{1}`: the city would silently disappear.

Checking translations automatically (run it in CI)

codeoutput/BundleCheck/BundleCheck.java — Java

```java
    public static void main(String[] args) throws IOException {
        Properties base = load(Path.of(args[0] + ".properties"));
        List<Path> files;
        try (Stream<Path> all = Files.list(Path.of("."))) {
            files = all.filter(p -> p.getFileName().toString().startsWith(args[0] + "_"))
                       .sorted().toList();
        }
        for (Path f : files) {
            Properties t = load(f);
            for (String k : new TreeSet<>(base.stringPropertyNames())) {
                if (!t.containsKey(k)) System.out.println(f + ": missing " + k);
                else if (values(t.getProperty(k)) != values(base.getProperty(k)))
                    System.out.println(f + ": " + k + " has " + values(t.getProperty(k))
                        + " values, base has " + values(base.getProperty(k)));
            }
            for (String k : new TreeSet<>(t.stringPropertyNames()))
                if (!base.containsKey(k)) System.out.println(f + ": unknown key " + k);
        }
    }
}
```

Checking translations automatically (run it in CI)

codeoutput/BundleCheck/BundleCheck.java — Java

```java
    public static void main(String[] args) throws IOException {
        Properties base = load(Path.of(args[0] + ".properties"));
        List<Path> files;
        try (Stream<Path> all = Files.list(Path.of("."))) {
            files = all.filter(p -> p.getFileName().toString().startsWith(args[0] + "_"))
                       .sorted().toList();
        }
        for (Path f : files) {
            Properties t = load(f);
            for (String k : new TreeSet<>(base.stringPropertyNames())) {
                if (!t.containsKey(k)) System.out.println(f + ": missing " + k);
                else if (values(t.getProperty(k)) != values(base.getProperty(k)))
                    System.out.println(f + ": " + k + " has " + values(t.getProperty(k))
                        + " values, base has " + values(base.getProperty(k)));
            }
            for (String k : new TreeSet<>(t.stringPropertyNames()))
                if (!base.containsKey(k)) System.out.println(f + ": unknown key " + k);
        }
    }
}
```

Terminal — codeoutput/BundleCheck.txt

```
$ java BundleCheck.java Messages
./Messages_de.properties: unknown key extra
./Messages_tr.properties: missing bye
./Messages_tr.properties: items has 1 values, base has 2
```

Smaller runtimes: `jlink --include-locales`

Terminal codeoutput/Jlink.txt

```
$ jlink --add-modules java.base --strip-debug --no-man-pages --no-header-files --output rt-base
$ jlink --add-modules java.base,jdk.localedata --strip-debug --no-man-pages --no-header-files --output rt-all
$ jlink --add-modules java.base,jdk.localedata --include-locales=en,tr --strip-debug --no-man-pages --no-header-files --output rt-en-tr
$ du -sh rt-base rt-all rt-en-tr
52M     rt-base
80M     rt-all
53M     rt-en-tr
$ rt-base/bin/java -cp classes Today
5 locales: tr-TR = 2026 Oct 15, Thu   de-DE = 2026 Oct 15, Thu
$ rt-all/bin/java -cp classes Today
1069 locales: tr-TR = 15 Ekim 2026 Perşembe   de-DE = Donnerstag, 15. Oktober 2026
$ rt-en-tr/bin/java -cp classes Today
114 locales: tr-TR = 15 Ekim 2026 Perşembe   de-DE = 2026 Oct 15, Thu
```

Only `java.base` has English data; all other locales are in `jdk.localedata`. Without it Turkish silently falls back to the *root* format. `--include-locales=en,tr` keeps exactly what you ship (German is gone).

Security

A real attack: the Turkish i

codeoutput/Register/Register.java — Java

```java
import java.util.Scanner;

public class Register {
    public static void main(String[] args) {
        String name = new Scanner(System.in).next();
        if (name.toUpperCase().equals("ADMIN")) {          // BUG: default locale
            System.out.println("rejected: reserved name");
            return;
        }
        System.out.println("created user '" + name + "'");  // on a Turkish
        server: admin!
    }
}
```

On a server whose default locale is Turkish, `"admin".toUpperCase()` is `"ADMİN"`: the check passes and an attacker owns the name `admin`. Security checks must use `Locale.ROOT` (or compare canonical keys).

A real attack: the Turkish i

codeoutput/Register/Register.java — Java

```java
import java.util.Scanner;

public class Register {
    public static void main(String[] args) {
        String name = new Scanner(System.in).next();
        if (name.toUpperCase().equals("ADMIN")) {          // BUG: default locale
            System.out.println("rejected: reserved name");
            return;
        }
        System.out.println("created user '" + name + "'");  // on a Turkish
        server: admin!
    }
}
```

Terminal — codeoutput/Register.txt

```
$ echo admin | java Register.java
rejected: reserved name
$ echo admin | LANG=tr_TR.UTF-8 java
      Register.java
created user 'admin'
```

On a server whose default locale is Turkish, `"admin".toUpperCase()` is `"ADMİN"`: the check passes and an attacker owns the name `admin`. Security checks must use `Locale.ROOT` (or compare canonical keys).

Look-alike user names

codeoutput/Usernames.java — Java

```
 4 public class Usernames {
 5     static String key(String name) {                    // canonical
       form for comparisons
 6         return Normalizer.normalize(name, Normalizer.Form.NFKC).
       toLowerCase(Locale.ROOT);
 7     }
 8     static String hex(String s) {
 9         StringBuilder b = new StringBuilder();
10         s.codePoints().forEach(c -> b.append(String.format("%X ", c)));
11         return b.toString().trim();
12     }
13     public static void main(String[] args) {
14         String[] tries = {"admin", "ADMIN", "\uFF41dmin", "ad\u217Fin",
           "\u0430dmin"};
15         for (String t : tries) {
16             String k = key(t);
17             System.out.printf("%-20s key=%-20s %s%n", hex(t), hex(k),
18                 k.equals("admin") ? "same as admin" : "different");
19         }
20     }
21 }
```

NFKC + `ROOT` lower case catches the full-width *a* (FF41) and the Roman numeral *m* (217F). The Cyrillic *a* (430) stays different: *confusables* need a separate check (Unicode TR 39, ICU `SpoofChecker`).

Look-alike user names

codeoutput/Usernames.java — Java

```
 4 public class Usernames {
 5     static String key(String name) {                    // canonical
           form for comparisons
 6         return Normalizer.normalize(name, Normalizer.Form.NFKC).
           toLowerCase(Locale.ROOT);
 7     }
 8     static String hex(String s) {
 9         StringBuilder b = new StringBuilder();
10         s.codePoints().forEach(c -> b.append(String.format("%X ", c)));
11         return b.toString().trim();
12     }
13     public static void main(String[] args) {
14         String[] tries = {"admin", "ADMIN", "\uFF41dmin", "ad\u217Fin",
            "\u0430dmin"};
15         for (String t : tries) {
16             String k = key(t);
17             System.out.printf("%-20s key=%-20s %s%n", hex(t), hex(k),
18                 k.equals("admin") ? "same as admin" : "different");
19         }
20     }
21 }
```

Terminal — codeoutput/Usernames.txt

```
61 64 6D 69 6E       key=61 64 6D 69 6E       same as admin
41 44 4D 49 4E       key=61 64 6D 69 6E       same as admin
FF41 64 6D 69 6E     key=61 64 6D 69 6E       same as admin
61 64 217F 69 6E     key=61 64 6D 69 6E       same as admin
430 64 6D 69 6E      key=430 64 6D 69 6E      different
```

NFKC + `ROOT` lower case catches the full-width *a* (FF41) and the Roman numeral *m* (217F). The Cyrillic *a* (430) stays different: *confusables* need a separate check (Unicode TR 39, ICU `SpoofChecker`).

More traps

Invisible and direction characters

Characters such as U+202E (right-to-left override) and U+200B (zero-width space) change how text *looks* without changing what it *is*: fake file names (`gpj.exe` shown as `exe.jpg`), hidden code in reviews (*Trojan Source*, CVE-2021-42574). Strip or reject them in identifiers.

Normalize, then validate

Validate **after** normalizing and decoding, never before: otherwise `..`, `/`, `<` can come back in another form. Keep one canonical form (NFC, UTF-8) in storage.

Untrusted patterns

A translation is code: `MessageFormat` patterns from outside can break formatting. Validate bundles (previous section) before deploying them.

Default locale in a data file

codeoutput/Csv/Report.java — Java

```java
public class Report {
    public static void main(String[] args) {
        // machine-readable output formatted with the DEFAULT locale: a bug
        System.out.println(String.format("%s,%.2f,%d", "tea", 12.5, 3));
    }
}
```

The Turkish server writes `12,50`: the CSV row now has 4 fields, and the next program reads the price as 12. Every machine format needs `Locale.ROOT`: `String.format(Locale.ROOT, ...)`, `toUpperCase(Locale.ROOT)`.

Default locale in a data file

codeoutput/Csv/Report.java — Java

```java
public class Report {
    public static void main(String[] args) {
        // machine-readable output formatted with the DEFAULT locale: a bug
        System.out.println(String.format("%s,%.2f,%d", "tea", 12.5, 3));
    }
}
```

Terminal — codeoutput/Csv.txt

```
$ java Report.java > en.csv
$ LANG=tr_TR.UTF-8 java Report.java > tr.csv
$ cat en.csv tr.csv
tea,12.50,3
tea,12,50,3
$ awk -F, '{ print FILENAME ": " NF " fields" }' en.csv tr.csv
en.csv: 3 fields
tr.csv: 4 fields
```

The Turkish server writes `12,50`: the CSV row now has 4 fields, and the next program reads the price as 12. Every machine format needs `Locale.ROOT`: `String.format(Locale.ROOT, ...)`, `toUpperCase(Locale.ROOT)`.

Encodings come from the environment

Terminal codeoutput/Encodings.txt

```
$ for l in en_US.UTF-8 tr_TR.UTF-8 tr_TR; do echo "== LANG=$l"; LANG=$l java -XshowSettings:properties -version 2>&1 | grep -E "^ +(file|native|
      stdout|sun.jnu).encoding"; done
== LANG=en_US.UTF-8
    file.encoding = UTF-8
    native.encoding = UTF-8
    stdout.encoding = UTF-8
    sun.jnu.encoding = UTF-8
== LANG=tr_TR.UTF-8
    file.encoding = UTF-8
    native.encoding = UTF-8
    stdout.encoding = UTF-8
    sun.jnu.encoding = UTF-8
== LANG=tr_TR
    file.encoding = UTF-8
    native.encoding = ISO-8859-9
    stdout.encoding = ISO-8859-9
    sun.jnu.encoding = ISO-8859-9
```

`file.encoding` (default charset) is UTF-8 since Java 18. `native.encoding`, `sun.jnu.encoding` (file names) and `stdout.encoding` still follow the operating system: a Latin-5 server writes Latin-5 to the console. Name the charset in every `Reader`, `Writer`, `getBytes`.

Delivery and Operations

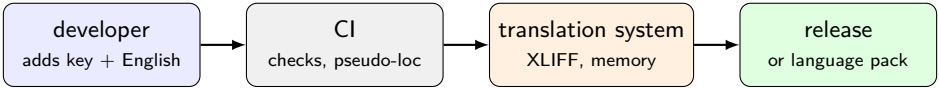
Servers and containers

```
FROM eclipse-temurin:21-jre
ENV LANG=C.UTF-8 TZ=UTC
COPY app.jar /app/app.jar
ENTRYPOINT ["java", "-Duser.language=en", \
  "-Duser.country=US", "-Duser.timezone=UTC", \
  "-jar", "/app/app.jar"]
```

Not run here. Same on **Windows** and **macOS** with Docker Desktop.

- Make the server's defaults **explicit**: a container without `LANG` differs from a laptop.
- The defaults should never matter: pass the user's locale and zone through the code.
- Keep the JDK updated: CLDR data and the time zone database come with it (`tzupdater` can patch only the zones).
- Log in English with `Locale.ROOT`; show the user a translated message with an error code.

The translation workflow



- Give translators **context**: a comment per key, a screenshot, the meaning of each `{n}`.
- Keys name the meaning (`order.items.count`), not the English words.
- One key per sentence; never reuse a key in two places (“Open” a file vs. “Open” as a state).
- Tools: XLIFF files, translation memories, platforms like Weblate or Crowdin; review in the running product.

Summary

Checklist for a product

Design

- locale-neutral core, formatting at the edge
- locale and zone per request / user, not global
- all machine formats with `Locale.ROOT`, UTF-8, ISO 8601
- language packs or reloadable bundles

Quality and security

- bundle checks and pseudo-localization in CI
- tests in `tr`, de, a right-to-left locale, several zones
- canonical keys (NFKC + ROOT) for identities
- explicit server defaults; JDK updates for CLDR and tzdata

Projects

1. Build a small web service (`com.sun.net.httpserver`) that answers in the language of `Accept-Language` and formats money and dates for it.
2. Turn the sophomore shop into modules with a German language pack provided as a separate module.
3. Extend `BundleCheck`: report apostrophe problems and `choice` patterns that do not parse; run it in CI.
4. Write a `ResourceBundle.Control` that reads translations from a database table and reloads every minute.
5. Audit an open-source Java project for `toUpperCase()`/`toLowerCase()`/`String.format` without a locale.
6. Measure the size and start-up time of `jlink` images with all locales vs. only yours.

Questions?